

Component Deep Dive: RGB-Gray-Blur-Sobel-Overlay Pipeline

Component owner(s): Lukas Röß

Top-level objective

[AXI_RgbGrayBlurrSobelOverlayPipeline](#) integrates the full real-time stream chain: RGB -> grayscale -> blur -> Sobel -> overlay compositor.

The module keeps one AXI4-Stream input/output interface and exposes runtime mode controls through debounced button logic.

Processing chain and submodule composition

Stage	Main module	Function in chain
Control	DebounceClickDetector	Generates stable mode bits for stage bypass and overlay style selection.
RGB split	AXI_RgbToGrayscale	Produces synchronized RGB base branch and grayscale processing branch.
Blur path	AXI_BlurrWindowModule	Optional low-pass stage on grayscale stream.
Sobel path	AXI_SobelWindowModule	Optional edge extraction on gray/blur stream; outputs RGB-formatted edge mask.
Merge	AXI_FrameCompositor	Aligns delayed base RGB with edge timing and composes final RGB output.

Table 1: Structural decomposition of [AXI_RgbGrayBlurrSobelOverlayPipeline](#).

Typical transaction sequence

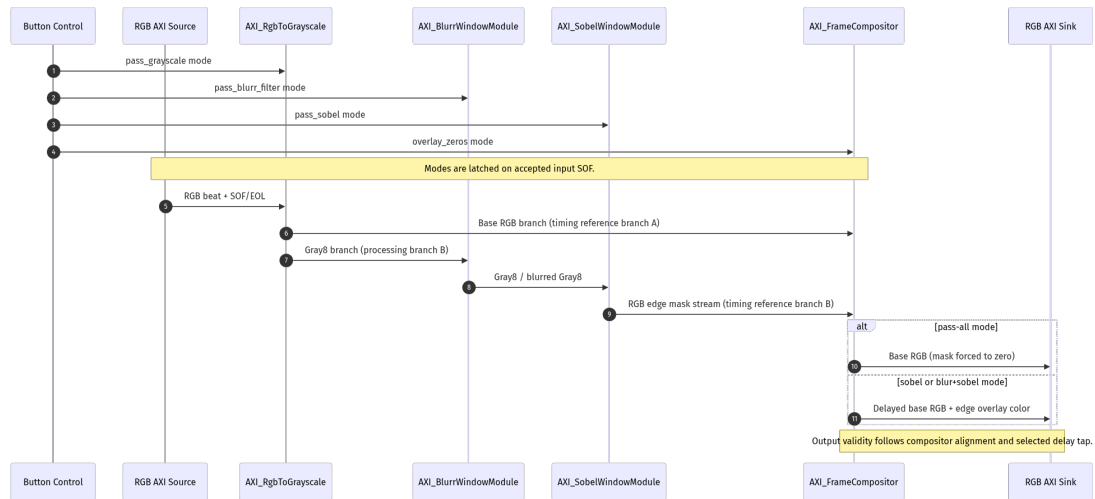


Figure 1: Integrated transaction sequence for [AXI_RgbGrayBlurrSobelOverlayPipeline](#).

Frame-consistent runtime control

The pipeline latches mode control values only at accepted start-of-frame beats:

$$\text{sof_accept} = \text{TVALID} \wedge \text{TREADY} \wedge \text{TUSER}$$

This is implemented in `s_input_sof_accept` and `P_REG_FRAME_CTRL_LATCH` (`rtl/PIPELINE/hdl/axi_rgb_gray_blurr_sobel_overlay_pipeline.vhd`). As a result, control changes do not tear a frame in the middle of active video.

Delay alignment and compositor coupling

The base RGB branch is always connected to the compositor input, while the Sobel branch provides timing and edge mask data ([rtl/PIPELINE/hdl/axi_rgb_gray_blurr_sobel_overlay_pipeline.vhd](#)). Delay tap selection is mode-dependent:

- C_DELAY_SEL_NONE for pass-through output,
- C_DELAY_SEL_SOBEL for grayscale+Sobel overlay,
- C_DELAY_SEL_BLUR_SOBEL for grayscale+blur+Sobel overlay.

These selections are generated in `s_fc_delay_stage_sel` ([rtl/PIPELINE/hdl/axi_rgb_gray_blurr_sobel_overlay_pipeline.vhd](#)) and consumed by `AXI_FrameCompositor` .

Sobel-threshold integration note

The top-level generic set exposes G_SOBEL_THRESHOLD ([rtl/PIPELINE/hdl/axi_rgb_gray_blurr_sobel_overlay_pipeline.vhd](#)) and forwards it to `AXI_SobelWindowModule` ([rtl/PIPELINE/hdl/axi_rgb_gray_blurr_sobel_overlay_pipeline.vhd](#)).

Historically, the project used a fixed Sobel threshold. The current Sobel core supports adaptive thresholding through running-mean parameters in `AXI_SobelFilter` and `E_SobelCore` . In the present top-level wrapper, these adaptive controls are not yet promoted to the pipeline generic interface.

Output ownership and ready propagation

The compositor is the final owner of `m_axis_video_rbg888_*`, and READY propagation follows this ownership:

- RGB branch READY is driven from compositor RGB READY.
- Sobel branch READY is driven from compositor gray READY.

This coupling is implemented at [rtl/PIPELINE/hdl/axi_rgb_gray_blurr_sobel_overlay_pipeline.vhd](#) and keeps both branches synchronized in accepted-beat space.

Verification with cocotb testbench

Pipeline-level verification is defined by:

- [tests.test_axi_gray_blurr_sobel_overlay_pipeline](#)
- [tests.test_axi_gray_blurr_sobel_overlay_pipeline_downscaled](#)
- target configuration in [testbench/targets.toml](#) and [testbench/targets.toml](#)

Visual stage progression on Lenna

Input RGB	Gray	Blur	Sobel	Overlay
image(".././../testbench/images/lenna_512_512.png", width: 100%)	image(".././figures/512_512_out_gray.png", width: 100%)	image("../figures/generated/blur.png", width: 100%)	image("../figures/generated/sobel_filter_output.png", width: 100%)	image("../figures/generated/pipeline_overlay_output.png", width: 100%)

Table 2: Pipeline stage sequence for one Lenna frame: original input, grayscale conversion, blur output, Sobel output, and final overlay composition.

Executed command for this report update:

```
cd testbench
uv run tb-sim --target axi_gray_blurr_sobel_overlay_pipeline_downscaled
```

Current status in this checkout:

- target result: FAIL
- root cause: missing source file [testbench/tests/vhdl/c_shift_ram_0_model.vhd](#)
- failure appears before simulation build (results.xml not generated for this run)

Even with this open issue, the pipeline test area already contains generated image artifacts from prior pipeline simulations under `testbench/sim_build/`. The latest available overlay artifact is shown below.



Figure 2: Latest available pipeline simulation artifact (`pipeline_full_chain_state3_sobel.png`) from `testbench/sim_build/test_axi_gray_blurr_sobel_overlay_pipeline/.../build/`.